

Glow / Optimization

AI-powered contingency acceleration UX case study

Role: UX/UI Designer · Enterprise energy software · MVP workflow and dashboard

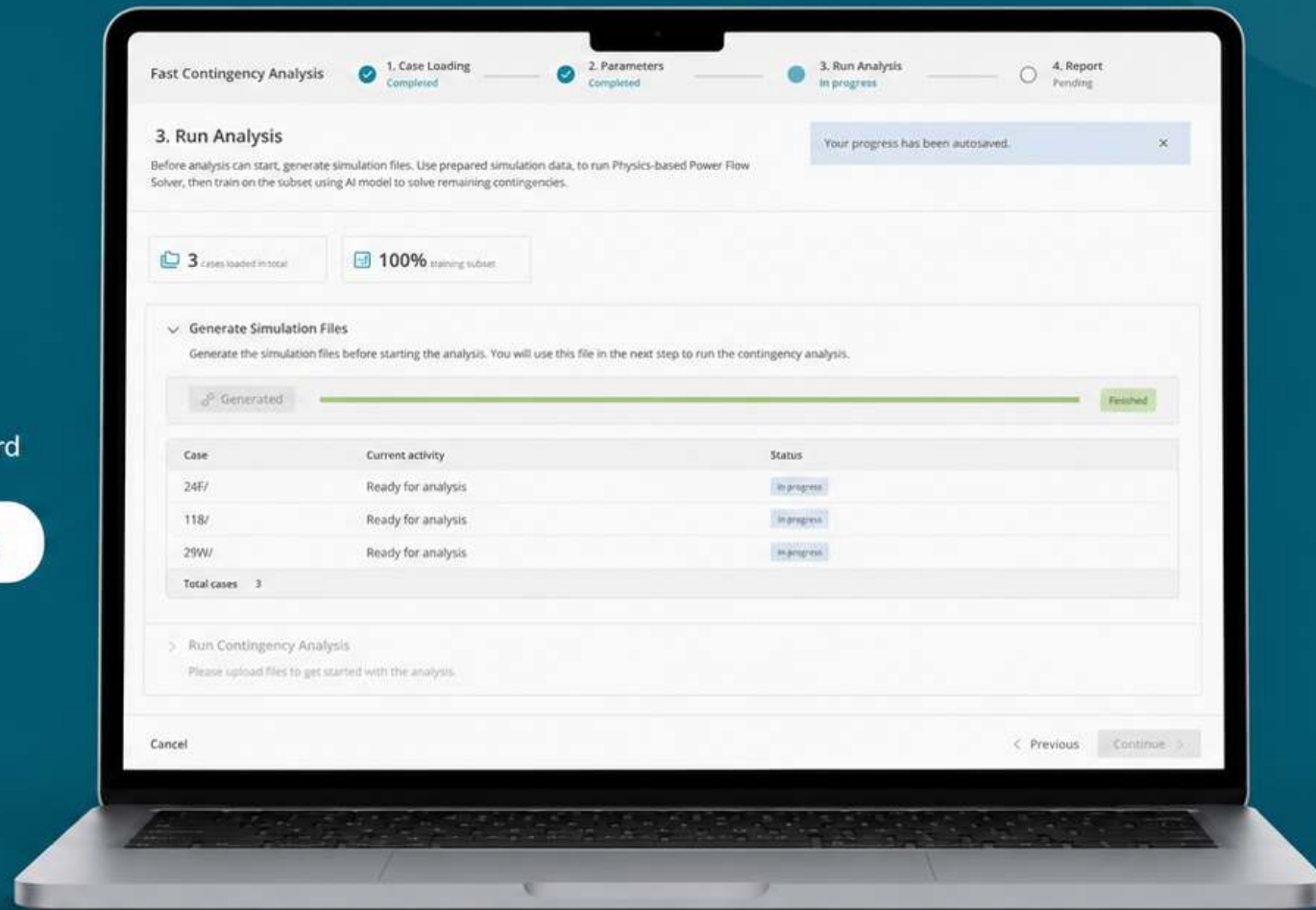
AI-assisted UX process

Hybrid solver + ML workflow

Senior UX framing

Portfolio version:

selected screens and workflow decisions are presented as a case study; sensitive implementation details can be adapted or anonymized before public sharing.



01. Product context: where I started

When I joined the work, the technical flow was already complex. My first job was to understand where that complexity could become confusing or unsafe for users.

What I learned about the product

- Glow supports contingency analysis for power-system planning and reliability scenarios.
- A physics-based power-flow solver provides ground truth on a subset of cases, while AI predicts the remaining contingencies.
- The workflow brings cases, parameters, generated files, run status and reports into one guided process.

What I saw as the UX risk

- The workflow crosses several systems and mental models, so it could easily feel like a black box.
- I needed users to see what is ready, what is blocked, what failed and what they can safely do next.
- Because this was an MVP, I also had to balance clarity with implementation effort and changing requirements.

02. Discovery: I started with user decisions, not screens

Because the product was still an MVP and not yet live, I used workshops, meeting notes and stakeholder input as the closest proxy for real user needs.

Who uses it—and what must they decide?

Define the primary user for release 1.0. Add or correct roles as needed.

Both of them can be sometimes the same person

Primary • Power-system engineer

Needs a fast, trustworthy view of contingency results without configuring ML details.

Decisions:

Which violations matter?
What should I inspect or export next?

Secondary • Data scientist / model owner

Needs fidelity and governance evidence comparing AI predictions with solver ground truth.

Decisions:

Is model quality acceptable? Where does it diverge?
Is retraining or deeper investigation needed? Can I trust this run?

Each participant

- 1) When you open this dashboard?
- 2) What question must it answer in the first 10 seconds?
- 3) What action should you be able to take next?
- 4) What should make the dashboard misleading or unusable?
- 5) Where would you see the status of all the configurations.
- 6) Where / how did I finished the process? Where can I resume?
- 7) What parameters / settings into configuration w/o not training wizard.

FigJam workshop framing

LIVE CAPTURE — one decision or need per sticky

Add a user need...

Add a decision...

Add a risk...

What I took from discovery

- I framed the power-system engineer as the primary user: someone who needs trustworthy results without having to manage ML details.
- I kept the model owner/data scientist as a secondary perspective because the same person may sometimes cover both roles.
- I repeatedly asked what the user needs to know in the first 10 seconds and what action they should be able to take next.
- Those answers shaped the status model, recovery paths and dashboard priorities before I polished the UI.

Stakeholder alignment

Decision capture

03. Information architecture: I treated naming as a product decision

When 'study', 'configuration' and 'analysis' started being used interchangeably, I did not want to fix it as a copy-only issue.



Study

Broad container; matches a complete saved workflow.
Risk: may feel generic if engineers do not use this term.



Configuration

Concrete and technical; already familiar in team discussions.
Risk: may sound like settings only.



Analysis

Action-oriented but weak for the dashboard if one saved item contains several case analyses.



Run

Good for execution state, but too narrow for drafts, setup and saved reports.

How I approached it

I mapped what each term could mean, compared the trade-offs, and brought the question back to the product team and Ozge for domain validation. I wanted the final UI to reflect the user mental model, not just whichever label appeared first.

04. Case loading: I tried to stop problems before the analysis starts

I wanted users to understand file readiness at a glance instead of discovering missing inputs later in the workflow.

Choose case folders. Each folder must contain one complete case with the following files: .raw, .mon, .con, .sub.

Reload

☐ 2023

☒ 2024

- ☒ 24E/
- ☐ 24F/
- ☒ 24G/

☐ 2025

☐ drafts

- ☒ 52A/

☐ 11B/

☐ 11N/

i Select up to 10 complete case folders.

Case selection and file validation

i 3/10 cases loaded in total

✓ 3 cases loaded successfully

! 0 missing files

Case	.raw	.mon	.con	.sub	Status	Notes
24E/	✓	✓	✓	✓	Complete	—
24E.raw	✓	—	—	—	—	—
24E.mon	—	✓	—	—	—	—
24E.con	—	—	✓	—	—	—
24E.sub	—	—	—	✓	—	—
24G/	✓	✓	✓	✓	Complete	—
52A/	✗	✓	✓	✓	Incomplete	Missing .raw
52A.raw	✗	—	—	—	—	Missing .raw
52A.mon	—	✓	—	—	—	—
52A.con	—	—	✓	—	—	—
52A.sub	—	—	—	✓	—	—

Case name
(folder name)

Required files

- Green check = file present
- Red X = file missing

Status

- Complete = all files present
- Incomplete = one or more files missing

Notes

- Shows which file is missing (if any)

Optional future enhancements

Summary of selection status

Table can also shown as a tree structure, where each case can expand to reveal its required files

Chevrons indicate expand/collapse behavior.

Why I designed it this way

- I split the screen between folder selection and validation so users can see cause and consequence together.
- I used status summaries to turn file-level checks into a quick readiness signal.
- I kept the tree/table pattern because engineers may still need to inspect exactly which file is missing.
- I avoided relying on tooltips for required guidance; the key rule needed to stay visible.
- I gated Continue until at least one complete case is selected, so the UI prevents an invalid next step.

Glow / Optimization UX case study

5/9

05. Run analysis: I focused on making the hybrid process explainable

My concern was not just whether the user could start the analysis, but whether they understood what the system was doing and why.

Fast Contingency Analysis

1. Case Loading
Completed

2. Parameters
Completed

3. Run Analysis
In progress

4. Report
Pending

3. Run Analysis

Before analysis can start, generate simulation files. Use prepared simulation data, to run Physics-based Power Flow Solver, then train on the subset using AI model to solve remaining contingencies.

3 cases loaded in total

100% training subset

Generate Simulation Files

Generate the simulation files before starting the analysis. You will use this file in the next step to run the contingency analysis.

Generate Files

Case	Current activity	Status
24F/	Waiting for files	Init
11B/	Waiting for files	Init
29W/	Waiting for files	Init
Total cases	3	

Run Contingency Analysis

Please upload files to get started with the analysis.

Generate files + start analysis

Cancel

Previous

Continue

Decisions I made

- I separated simulation-file generation from the analysis step so users can see what is blocking progress.
- I worked through the solver handoff and confirmation wording with the team instead of hiding the external step behind generic copy.
- I kept case-level statuses tied to backend/file-generation state rather than mixing them with solver results.
- For partial failures, I pushed for the failed case to stay visible and explain why no report will appear, while successful cases can continue.

Trust through status

No black box

06. Dashboard: I designed for returning users, not just first entry

I treated the dashboard as the place users come back to recover context: where they stopped, what finished, and what needs attention.

Welcome to Contingency Acceleration

Start a new study or resume an existing one. A study contains one or more cases representing different system scenarios. Add cases, define parameter configurations, and manage or continue previously saved studies.

+ Start

Your Configurations

View and continue your existing analysis configurations.

Configuration name	Cases included	Status	Created	Action
Summer Peak 2026	3	In progress	21 May 2026, 14:32	> Resume
Winter Peak 2025	10	Finished	20 May 2026, 09:15	> Open Report
Test Grid A	2	Idle	18 May 2026, 16:45	> Continue Setup
Pilot Run	8	Error	17 May 2026, 11:10	> Review Issue
Spring Peak 2026	5	In progress	16 May 2026, 13:25	> Resume
Autumn Peak 2025	7	Finished	15 May 2026, 10:40	> Open Report
Grid Validation	4	Idle	14 May 2026, 16:10	> Continue Setup
Baseline Study	6	Error	13 May 2026, 09:30	> Review Issue
North Region Test	9	In progress	12 May 2026, 15:05	> Resume
South Region Test	10	Finished	11 May 2026, 08:50	> Open Report

10 ▼ of 290 rows |< < 1 / 29 > >|

Each configuration represents one set of up to 10 cases and its analysis.

How the design evolved

- I mapped the action to the state: Resume, Open Report, Continue Setup or Review Issue.
- When I learned the backend only stores creation time, I changed Last updated to Created instead of implying data we do not have.
- I first considered normal page scrolling, but once the implementation already had 290+ saved items, I changed the recommendation to pagination with 10 rows by default.
- I added empty, loading and load-error states so no data is not confused with data failed to load.
- I deliberately left search/filtering out of the MVP because it had not yet been discussed or validated.

07. How I used AI in my own design process

I used AI to speed up synthesis and exploration, but I kept the product judgement, stakeholder alignment and final UX decisions with me.

AI

Meeting intelligence

I used Claude and Copilot to turn long meeting transcripts into decisions, open questions, risks and follow-ups I could bring back to the team.

AI

Microcopy exploration

I used AI to explore wording for technical solver/AI concepts, then narrowed the options based on domain meaning and the surrounding UI.

AI

Design acceleration

I used Figma Make and AI-assisted prototyping to explore layout directions quickly, then refined the strongest option manually in Figma.

AI

Quality control

I used AI as a second pair of eyes for edge cases, terminology consistency, dashboard states and handoff checks.

How I kept ownership

I treated AI as an accelerator, not an authority. If an output conflicted with the product model, stakeholder knowledge or the user task, I changed or discarded it.

08. What I learned from the project

The biggest shift for me was seeing that the hardest part of an AI-enabled enterprise product is often not the model itself, but helping people understand and trust the workflow around it.

UX

What I delivered

- I structured the MVP wizard around user decisions rather than technical steps alone.
- I mapped statuses, confirmations and failure states across the flow.
- I translated solver/AI complexity into clearer user-facing steps.

UX

How my thinking evolved

- I stopped treating terminology as a copy detail and started treating it as part of the product model.
- I changed recommendations when implementation facts changed, rather than defending an earlier UX idea.
- I separated MVP needs from useful but unvalidated enhancements.

UX

What I would carry forward

- In AI software, trust needs visibility, control and understandable status.
- In domain-heavy tools, I bring naming questions back to domain experts instead of assuming UX has the answer.
- AI helps me most when it improves synthesis and iteration, not when it replaces design accountability.

My takeaway

I am comfortable working where UX strategy, AI workflows, domain-heavy enterprise software and delivery constraints meet - and I can still keep decisions grounded in users, evidence and stakeholder context.